TECHNICAL REPORT | OCTOBER 2026

litresearch

An iterative, state-centric agent for autonomous literature research

Gihan Uthpala gihanuthpala@gmail.com

ABSTRACT

Most retrieval-augmented systems treat literature search as a single retrieval step followed by summarization. Real literature research works differently: the investigator's understanding of the question changes as they read, new subquestions appear, and evidence is weighed, not just collected. We present **litresearch**, an open-source agent that runs literature research as an *iterative evidence-acquisition process* over an explicit, mutable research state. A research question is broken into a tree of subquestions, each served by *search intents* that are translated into source-specific queries over arXiv, OpenAlex, Crossref, PubMed, Semantic Scholar and the web. Results are merged into a canonical, provenance-preserving document registry through a conservative four-tier deduplication hierarchy. The registry is expanded by citation snowballing until a saturation criterion is met, and then screened, ranked and allocated to subquestions round-robin. Documents are read in parallel into a structured *evidence store* of claims with support, location, type and strength. A critic reviews the state along eight dimensions and returns *research actions* rather than a binary verdict: it can add, split, rephrase, deprioritize or close subquestions, so the research plan is an output of the investigation as well as an input. The final report is written section by section from the evidence store, and every citation resolves to a retrieved document. We describe the design, the deterministic machinery that keeps a language-model pipeline auditable, and a case study that read 28 of 237 retrieved documents in about three and a half minutes. We also report weaknesses the case study exposed.

Keywords LLM agents · literature review automation · retrieval-augmented generation · citation snowballing · evidence synthesis · LangGraph

01 Introduction

Large language models (LLMs) combined with retrieval have become a standard way to answer knowledge-intensive questions [9]. For *literature research*, meaning the task of establishing what a body of scholarly work says about a question, a single round of retrieve-then-summarize falls short in three ways.

First, the right decomposition of a research question is rarely clear at the start. It becomes clear while reading: a paper introduces terminology the investigator did not know, shows that a subquestion has two parts that need separate evidence, or shows that another subquestion is a dead end. Second, the output of reading should be *evidence*, not summaries. A summary turns a paper into a single opinion. Evidence keeps each claim separate from what supports it, so a later stage can ask whether a claim is established by several independent studies or asserted by one. Third, systematic reviewers have long known that keyword search alone misses relevant work, and that following citations backward and forward is needed

for coverage [6, 17].

litresearch is built around these observations. Its central design principle is that **the research plan is not fixed**: subquestions, intents, documents, evidence, contradictions and coverage estimates all live in one explicit state, and every stage of the pipeline reads and changes that state. The contributions of this paper are:

Contributions

1. A **state-centric architecture** for literature research, in which a five-level hierarchy (question → subquestion → intent → query → source) is first-class, mutable data. Its reducers remain correct under parallel fan-out (Section 4.1).
2. A **conservative canonicalization hierarchy** for scholarly records gathered from heterogeneous sources. It uses numeral vetoes, Unicode folding and verified identifier lookups, so false merges, which silently destroy evidence, are rare (Section 6).
3. **Deterministic ranking, allocation and saturation rules** that limit LLM calls to the points where judgement is needed. These include a round-robin reading allocator that prevents one subquestion from using up the reading budget, and a direction-aware snowballing policy (Sections 7 and 8).
4. A **structured evidence store** and a **critic that returns plan edits**, which together close an adaptive research loop whose termination is controlled by both quality and budget (Sections 9 and 10).
5. An **end-to-end case study** that reports what the system did, including several failure modes found in the process (Section 13).

02 Background and Related Work

Retrieval-augmented and agentic LLMs. Retrieval augmentation conditions generation on retrieved passages [9]. Agentic variants interleave reasoning with tool use [18] or with retrieval at each reasoning step [15, 16]. Self-reflection methods have the model critique and revise its own output [10, 14]. **litresearch** uses reflection at the level of the *research plan* rather than the generated text: the critic's output is a set of typed edits to the subquestion tree and new search intents, and these are applied to the state.

LLM systems for scholarly synthesis. PaperQA [7] answers scientific questions by retrieving and reading full-text papers. OpenScholar [1] builds a retrieval datastore over open-access literature for citation-backed synthesis. STORM [13] uses perspective-guided question asking to outline long-form articles. Work on citation-grounded generation [4] shows that LLMs often cite sources that do not support the claim. **litresearch** addresses this structurally. Citations are document identifiers taken from the evidence passed to each report section, unresolvable identifiers are removed, and the reference list is built from the document registry rather than generated by the model.

Systematic review methodology. Kitchenham's procedures [6] and the PRISMA statement [11] formalize search, screening, eligibility and synthesis for human-conducted reviews. Wohlin [17] describes backward and forward snowballing as an iterative procedure that runs until no new relevant papers are found. **litresearch** implements an automated, budgeted version of this cycle. Its saturation criterion directly operationalizes "no new papers."

Record linkage. Deduplicating bibliographic records is an instance of record linkage [2, 3]. Our hierarchy puts persistent identifiers first and fuzzy matching last, and it is tuned for precision over recall. The reason is specific to this setting: a missed duplicate costs one redundant read, while a false merge silently attributes one paper's evidence to another.

03 Design Principles

Five principles shaped the implementation.

P1. Research is a state, not a pipeline.
The state model, rather than the graph topology, encodes the method. Every node is a function from state to a *delta* on state.

P2. The plan is mutable.
Subquestions carry status, priority, coverage and origin (`planner`, `critic` or `discovery`), and the critic can rewrite them.

P3. Evidence, not summaries.
The unit of output from reading is a claim paired with its support, location, type and strength.

P4. Use LLMs only for judgement.
Ranking, reading allocation, snowball seed selection, saturation, deduplication, termination and reference assembly are deterministic code, so they are testable and auditable.

P5. Budgets are limits, not targets.
The number of documents retrieved is never a stopping criterion. Budgets cap cost, and the critic decides sufficiency.

04 Architecture

litresearch is implemented as a LangGraph [8] state graph (Figure 1). It contains two loops with different jobs. The **inner loop** (snowball ↻ screening) expands the citation neighbourhood of documents already found and stops on a saturation criterion. The **outer loop** (critique ↻ intents) is the adaptive research loop, and it can rewrite the research plan.

4.1 The research state

The state records the investigation as a hierarchy:

$$\underbrace{\text{question}}_{\text{fixed}} \rightarrow \underbrace{\text{subquestions}}_{\text{tree, mutable}} \rightarrow \underbrace{\text{intents}}_{\text{what to find}} \rightarrow \underbrace{\text{queries}}_{\text{per source}} \rightarrow \underbrace{\text{documents}}_{\text{canonical}} \rightarrow \underbrace{\text{evidence}}_{\text{claims}}.$$

The two middle levels are deliberately separate. A *subquestion* states what we need to *know*. A *search intent* states what kind of document would close that gap, for example "empirical comparisons of episodic versus semantic memory on long-horizon benchmarks." Because they are separate, query wording can be rewritten freely without changing the research objective it serves. Each intent also records its own yield (documents found, and how many screened as relevant), which is the signal the critic uses to judge whether an intent is working.

Reducers under fan-out. Parallel graph branches write to shared channels, so each channel has a merge function (a *reducer*) that must be associative and must tolerate two branches having seen the same baseline. Three rules make this work.

- **Budgets are written as deltas.** Each reader writes `documents_read=1` and the reducer sums them, so twenty parallel readers advance the counter by exactly twenty. If branches wrote running totals, each one would need to know what the others had done.
- **Documents merge field by field.** The more informative value wins: the longer abstract, the union of sources and provenance, the maximum of each subquestion's relevance score, and the minimum

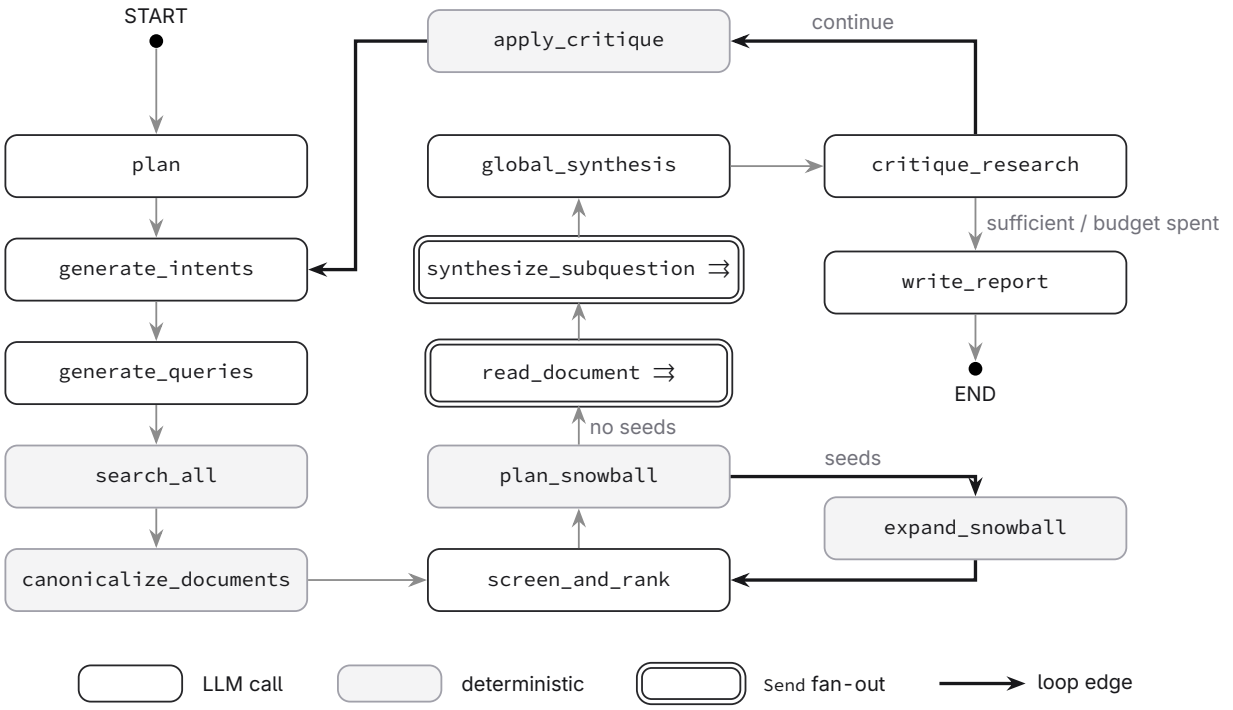


Figure 1 The research graph. White nodes call an LLM and grey-filled nodes are deterministic. Heavy black edges are loops. Double-bordered nodes fan out with LangGraph's `Send`, one branch per document or subquestion. The inner loop (right, lower) expands the citation graph and re-screens. The outer loop (top) applies the critic's research actions and re-enters at intent generation, not at planning, so the existing plan is edited rather than regenerated.

snowball depth. A sparse web hit therefore never degrades a rich record.

- **Document status forms a lattice.**

$$\texttt{candidate} \prec \texttt{selected} \prec \texttt{skipped} \prec \texttt{failed} \prec \texttt{read}.$$

Merging takes the join. `read` dominates, so a completed read never regresses. `failed` outranks `selected`, so a failed read is recorded as a failure and the document is not re-queued in every later iteration.

Per-iteration work lists (query plans, pending hits, the reading queue) replace their previous contents rather than accumulating, so an iteration starts from a fresh query set. Evidence and contradictions are append-only, with duplicates removed by content hash.

4.2 Model routing

Each LLM role (planner, intent, query, screener, reader, synthesizer, critic, writer) is routed independently to any `provider:model` specification. The roles differ in cost by orders of magnitude. Screening and reading run once per *document* (or batch of documents), while planning and critique run once per *iteration*. Per-role routing is therefore the main cost control: a capable model on the judgement roles and a cheaper one on the high-volume roles. All structured calls go through one helper that validates output against a Pydantic schema and returns `None` on failure. Each node then uses an explicit fallback, so a malformed response degrades a single step instead of stopping the run.

05 Planning and Retrieval

Table 1 Literature sources. "Citation graph" is the direction of snowballing that the source supports.

Source	Key	Citation graph	Best for
arXiv	—	none	very recent CS/ML preprints
OpenAlex [12]	—	both	broad coverage, DOIs, snowballing
Crossref	—	backward	deposited reference lists
PubMed	optional	none	biomedical literature
Semantic Scholar [5]	effectively required	both	richest citation graph
Web (Tavily)	required	none	grey literature, documentation

Decomposition. The planner is instructed to produce the *minimum* set of meaningful subquestions, each with a rationale and a priority. Because each intent's yield is recorded, the value of decomposition can be measured: setting `max_subquestions` = 1 gives a single-query baseline.

Source routing. A single query string cannot serve every scholarly API. arXiv uses literal keyword matching with terms combined by AND, Semantic Scholar ranks natural-language queries semantically, and PubMed rewards controlled vocabulary. **litresearch** therefore first *selects* the sources relevant to an intent and then generates a query specialized for each, up to $\min(4, |\text{sources}|)$ queries per intent. Each (source, query) pair counts as one search operation against the budget. Intents from higher-priority subquestions are planned first, so if the budget runs out, the least important work is the work that is lost. Intents that would produce no new queries are closed.

Sources. Table 1 lists the supported sources. arXiv, OpenAlex, Crossref and PubMed need no key. Semantic Scholar has three settings (`auto`, `on`, `off`), and under `auto` it is used only when an API key is present. Its unauthenticated pool is shared per IP address and returns HTTP 429 even under light load, so leaving it on without a key spends search budget for nothing. HTTP search runs concurrently inside a single node with a semaphore, rather than as one graph branch per query. Each source already enforces its own rate limit (arXiv asks for one request every three seconds), and per-query branches would bloat the state without any benefit.

06 Canonicalization

A single paper often arrives from several sources: arXiv as a preprint, Crossref with the published DOI, OpenAlex with a citation count, and the web as a landing page. These records are merged into one canonical document *before* reading, which is the expensive stage, and every `Provenance` record (subquestion, intent, source, query, rank, and whether the document was found by search or by snowballing) is kept. This means a paper retrieved for one subquestion can serve another without being retrieved again, and retrieval coverage can be evaluated afterwards.

Matching hierarchy. Candidates are matched in order of decreasing strength:

1. DOI (normalized);
2. a persistent identifier: arXiv (version suffix stripped), PubMed, OpenAlex or Semantic Scholar;
3. normalized title plus first-author surname;
4. title similarity of at least 0.93 (`SequenceMatcher` ratio) together with a corroborating signal (year or authors).

Tier 4 is the only fuzzy tier, and it is intentionally strict. The threshold of 0.93 is high because papers in a series ("… Part II", "*Method* v2") score between 0.85 and 0.92 and must not be merged.

Numeral veto. Numbers in a title almost always distinguish separate works: *Llama 2* and *Llama 3*, *GPT-3*

and *GPT-4, Part II* and *Part III*. These pairs differ by one or two characters and would pass any similarity threshold. We therefore compute a *numeric signature* (the sorted multiset of Arabic and Roman numerals in the title), and any mismatch vetoes a fuzzy merge.

Truncation. A record may carry a short title or drop a subtitle. A prefix match is accepted only when the shorter title has at least three distinctive tokens, because no length threshold separates a specific stub ("llm planner") from a generic one ("deep learning"). Containment, meaning all distinctive tokens of the shorter title appear in the longer one, requires at least four.

Unicode folding. Names and titles are NFKD-normalized, combining marks are removed, and dash variants are unified before comparison. This fixed a real failure: OpenAlex writes *Romera-Paredes* with U+2010 HYPHEN, which compared as "paredes" against "romera-paredes" and caused a correct match to be rejected. A full title match is also *never* vetoed because the author lists do not overlap. Sources render names too inconsistently (initials only, collective authors, transliterations) for that signal to override a matching title. Author agreement is required only when the title evidence is weak.

Verified identifier lookups. Some arXiv DOIs in OpenAlex are attached to the wrong record. Looking up `10.48550/arXiv.2201.11903` (Chain-of-Thought prompting [16]) returns an unrelated paper. Snowballing from that record would add someone else's references to the corpus under the seed's name. Every OpenAlex identifier lookup is therefore checked against the document's title and authors, with a punctuation-stripped title search as a fallback. A lookup that cannot be verified is rejected and logged.

07 Screening, Ranking and Reading Allocation

Screening. An LLM scores batches of documents (ten by default) against every active subquestion and returns a relevance score $r_q(d) \in [0,1]$ and a one-line note for each document. The prompt tells the model to judge substance rather than keyword overlap, since shared terminology is the most common cause of a bad screening decision. Scores are merged by maximum. Subquestion identifiers returned by the model are *repaired* before matching. Some models echo the whole prompt line ("`RQ1: How do agents...`") where the bare id `RQ1` is expected, and discarding those entries would score every document zero. When a new subquestion is added, the existing corpus is screened only against it, so document–subquestion pairs that already have scores are not paid for twice.

Ranking. The rank score combines relevance with the retrieval objectives from the methodology:

$$s(d) = \lambda(d)\Big[w_r \max_q r_q(d) + w_c C(d) + w_t T(d) + w_s Q(d) + w_m \min\big(\tfrac{|S(d)|}{3}, 1\big) + w_p P(d) + w_n N(d)\Big], \quad (1)$$

where $S(d)$ is the set of sources that returned d, $P(d) \in \{0,1\}$ is a heuristic for primary research, and $\lambda(d) = 0.6$ when d has no abstract in abstract-only mode (1 otherwise). The default weights are $w_r = 1.0$, $w_c = 0.15$, $w_t = 0.20$, $w_s = 0.10$, $w_m = 0.10$, $w_p = 0.10$ and $w_n = 0.15$. Relevance dominates by design, and the other terms act as tie-breakers. Citation impact is normalized by the paper's age:

$$C(d) = \min\left(\frac{\log(1 + c_d/a_d)}{\log 61}, 1\right), \qquad a_d = \max(1,\ y_{\text{now}} - y_d + 1), \quad (2)$$

so a paper with sixty citations per year saturates the term. Raw citation counts would push down work from the last year or two, which is exactly the work a question about recent developments needs. A missing citation count scores 0.3 rather than zero, since missing data is not evidence of missing impact. Recency $T(d)$ is 1 for papers at most one year old and falls linearly to 0 at fifteen years. Source quality $Q(d)$ is 1.0 for a DOI with a venue, 0.85 for other indexed records, 0.7 for preprints without a DOI, and 0.35 for web-only records. Novelty is one minus the largest title-token Jaccard similarity to any document already read,

$$N(d) = 1 - \max_{d' \in \mathcal{R}} J\big(\tau(d), \tau(d')\big), \quad (3)$$

Algorithm 1: Round-robin reading allocation

Input: ranked corpus $\mathcal{D}$; active subquestions Q sorted by priority; threshold θ; iteration cap K; per-subquestion cap k
Output: assignments $A : d \mapsto$ set of subquestions

foreach $q \in Q$ **do**
  $L_q \leftarrow [\, d \in \mathcal{D} : r_q(d) \geq \theta,\ q \notin \mathsf{readFor}(d) \,]$ sorted by $0.6\, r_q(d) + 0.4\, s(d)$ descending;
selected $\leftarrow \emptyset$;
repeat ▷ phase 1: costs budget
  foreach $q \in Q$ *with* $|\{d : q \in A(d)\}| < k$ **do**
    take next $d \in L_q$ with $q \notin A(d)$; $A(d) \leftarrow A(d) \cup \{q\}$; *selected* $\leftarrow$ *selected* $\cup \{d\}$;
until $|$*selected*$| \geq K$ *or no subquestion advanced*;
foreach $q \in Q$ **do** ▷ phase 2: free attachments
  foreach $d \in L_q \cap$ *selected while* $|\{d : q \in A(d)\}| < k$ **do**
    $A(d) \leftarrow A(d) \cup \{q\}$;
return A, *ordered by* $s(d)$ *descending*;

which keeps the reading queue from filling with near-duplicates once an area is well covered.

Reading allocation. A single global top-N cut lets one well-served subquestion take the whole reading budget and leave nothing for the others, which works against coverage. **litresearch** instead allocates reads round-robin across subquestions in priority order (Algorithm 1). A second phase then attaches every other subquestion that a selected document is relevant to. Reading one document against several subquestions is a single LLM call, so these attachments cost no reading budget. This is how a paper retrieved for RQ1 also comes to serve RQ3. The queue is sorted by rank score, so if the budget runs out partway through, the least valuable reads are the ones lost.

08 Citation Snowballing

Snowballing runs after ranking and starts only from documents that screened as clearly relevant ($r_q \geq 0.7$) and were found by direct search. Seeds are chosen without an LLM call, because the shape of the corpus already indicates which *direction* is useful:

- If a subquestion's relevant evidence is clustered in recent years (a span of at most 3 years with median year ≥ 2022), its foundations are missing. The system expands **backward** from the most relevant recent paper, whose reference list is a curated path into the area's earlier work.
- If the median year is old (≤ 2020), or the corpus is mixed, recent developments are missing. The system expands **forward** from the most-cited paper, which has the most citing work to find.

Seeds explicitly requested by the critic take precedence over automatic ones. Each source supports a different chain of directions:

$$\text{backward: S2}^{\dagger} \rightarrow \text{OpenAlex} \rightarrow \text{Crossref}, \qquad \text{forward: S2}^{\dagger} \rightarrow \text{OpenAlex},$$

where † marks Semantic Scholar, used only with a key. OpenAlex comes first because each edge it returns is a full record with an abstract, so the candidate can be read. Crossref often returns *more* backward edges (43 against 22 for one IEEE paper), but roughly half lack a DOI. Its DOI-bearing entries are therefore looked up in OpenAlex in batches of 50 to recover titles, authors and abstracts.

Saturation. Following Wohlin's stopping rule ("until no new papers are found") [17], round k is assigned a yield

$$\rho_k = \frac{\#\,\text{new canonical documents in round } k}{\#\,\text{candidates examined in round } k}, \tag{4}$$

measured in *new canonical documents* rather than raw hits. A round that returns 200 references already in the corpus has saturated, even though it returned plenty of data. Snowballing stops when two *consecutive* rounds satisfy $\rho < \rho_{\min}$ (default $\rho_{\min} = 0.10$), when the round budget is used up, or when the reading budget is used up. Requiring two rounds keeps a single seed with poor citation coverage from ending exploration too early. Snowballed documents re-enter screening and are ranked on the same terms as searched ones.

09 Reading and the Evidence Store

Each document in the reading queue is read in its own `Send` branch, against all subquestions assigned to it, in a single call. The reader's main output is a list of evidence items (Listing 1). A good claim is a single, self-contained, checkable assertion that names the method, the baseline and the task. "Our method outperforms the baseline" is useless once separated from the paper.

```
{ "claim":         "<method> improves <metric> on <benchmark> from X% to Y% over <baseline>",
  "support":       "Table 3: ...",            "location": "Section 4.2",
  "evidence_type": "experimental",            "strength": 0.55,
  "subquestion_id":"RQ3",  "doc_id": "doc_0050",  "is_limitation": false }
```

Listing 1 An evidence item (fields abridged).

Strength is a judgement about evidence, not about assertion. The reader is told to score strength low for single-benchmark results presented as general, missing baselines, missing ablations, unstated sample sizes, and results visible only in an abstract. In the default abstract-only mode it must set `location` to "Abstract" and keep strength at or below 0.6, since it cannot verify the experimental setup. With `--full-text`, PDFs are fetched and split into sections, so locations refer to real sections. Each evidence item is attributed to exactly one subquestion. A finding that serves two subquestions is emitted twice, with the claim phrased for each.

Claim clusters and support. The flat, append-only evidence list is grouped into *claim clusters* keyed on normalized claim text. A cluster is *weakly supported* when

$$\left|\{\mathrm{doc}(e) : e \in \mathrm{cluster}\}\right| < 2 \quad \text{or} \quad \overline{\mathrm{strength}} < 0.5. \tag{5}$$

This is the distinction between "the literature shows X" and "one paper claimed X," and the critic acts on it. Clustering on exact text is a deliberate lower bound. Reconciling semantically equivalent claims needs judgement, so it is left to the subquestion synthesizer.

Diversity. A *diversity report* is computed over documents that actually contributed evidence, not over everything retrieved: a corpus of 200 papers in which eight supply all the evidence is concentrated. It records the number of distinct first authors and venues, the share of evidence supplied by the top document and by the top author group, the preprint and grey-literature shares, the median year, and the share from the last three years. It flags *concentration* when the top document supplies more than 35% of the evidence, the top author group more than 30%, or there are fewer than four distinct first authors.

10 Synthesis, Reflection and the Adaptive Loop

Synthesis. Subquestion synthesis fans out, one branch per subquestion that has evidence. Each branch combines its evidence into an answer, states what the evidence does *not* establish, and scores its own coverage. Global synthesis then reads those answers and builds the cross-cutting narrative and the report

Table 2 Plan edits the critic can issue. Applying them is deterministic.

Edit	Effect on the subquestion tree
`add`	New top-level subquestion with origin `critic` (default priority 0.7).
`split`	Parent marked `superseded` and keeps its evidence; children `RQ`*n*`.1`, `.2`, ... inherit its priority and receive new searches.
`rephrase`	Question text replaced, typically to adopt terminology the literature uses.
`deprioritize`	Status `deprioritized`, priority capped at 0.2; excluded from mean coverage.
`mark_answered`	Status `answered`; no further searches are directed at it.

outline. It derives that outline from the findings rather than repeating the initial subquestion list, because the right structure for a literature review usually becomes clear only after the literature has been read.

The critic. The critic assesses eight dimensions: coverage, evidence quality, citation coverage, source diversity, contradictions, recency, knowledge gaps, and search saturation. Its inputs include the weakly supported claim clusters, the diversity report, and the yield history of each snowball round and each intent. It does not return a binary verdict. It returns *research actions*: new search intents, snowball requests, newly detected contradictions, and plan edits (Table 2). Knowledge gaps stated as free text are attached to the subquestion they most likely concern by token overlap. Ties are broken toward subquestions the critic itself flagged as weak, then toward the one with the lowest coverage.

Termination. The critic decides whether the evidence is *sufficient*. The budget decides whether the run may *continue*, because a critic that keeps asking for one more round still has to be stopped. The decision follows a fixed order:

1. no critique was produced ⇒ report (defensive stop);
2. the critic judges the evidence sufficient ⇒ report;
3. the iteration, reading or search-operation budget is used up ⇒ report;
4. the critic asks to continue but proposes no actionable intent, snowball request, `add` or `split` ⇒ report, because otherwise the loop would spin without changing the state;
5. otherwise, apply the critique and return to intent generation.

The run records a readable reason for stopping, which is included in the report.

11 Report Generation

The report is written section by section, following the outline produced by global synthesis. Each section receives only the evidence that belongs to it. This keeps citations accurate and stops a long report from degrading toward the end. Documents are cited inline by registry identifier (`[doc_0012]`). After generation, any identifier not in the registry is removed from its bracketed group, and the reference list is built from the documents actually cited in the body. A citation can therefore point only to a document that was retrieved. Some sections are produced deterministically from the state rather than by the model: the scope and method section, the reading guide, the source-diversity caveats, contradictions, limitations, and the reason the run stopped. If no evidence was collected, a diagnostic report explains why instead of an empty synthesis.

The run produces a browsable corpus on disk: the research plan, the full state as JSON (which `--resume` reads), a document index, per-document metadata with provenance and reading notes, per-subquestion syntheses, the evidence store grouped by claim cluster, contradictions, and the final report. The state is checkpointed after every graph step, so an interrupted run leaves a usable partial corpus.

12 Implementation

litresearch is about 9,500 lines of Python (3.11+) on LangGraph, LangChain's provider-agnostic `init_chat_model`, Pydantic and httpx, with Anthropic, OpenAI, Google and Ollama back-ends available as optional extras. Configuration has four layers, lowest priority first: code defaults, a committed `litresearch.yaml` (merged key by key along the directory path), `.env` for secrets, and environment variables, which the CLI's flags also set.

Local models. A fully local configuration needs no API keys. We found three factors that decide whether a local model works here, and parameter count matters least. (i) Structured output is constrained to the JSON schema during decoding, so output always parses and the question is whether its *content* is good. (ii) Ollama's default context window is 4,096 tokens, and it truncates silently. A screening prompt containing ten abstracts exceeds that, so the context size is set on every call. (iii) Mixing models across roles makes Ollama unload and reload weights between calls, which dominates runtime, so every role should use one model. Among the models we tested, `gemma4` produced critiques grounded in the actual state. `gpt-oss` passed a screening test but judged an obviously relevant paper irrelevant during extraction and returned no evidence, a failure that a general benchmark would not have revealed.

Testing. 250 offline tests cover the deterministic machinery: the deduplication hierarchy, reducers under fan-in, ranking and allocation, snowball saturation, source parsing against recorded API responses, and corpus round-tripping. They also run the whole graph end to end against a fake model and fake sources. The tests do not measure the quality of model output. For that, a separate script runs a candidate model against the real schemas and prompts and flags the failure modes that matter: keeping a deliberately off-topic paper, rejecting a relevant one, and copying search intents from the prompt's own examples. Another script checks the source adapters against the live APIs. Scholarly APIs reject the entire request when one field is wrong, so a misspelled field name disables a whole source while every offline test still passes. Three real adapter bugs were found this way.

13 Case Study

We ran **litresearch** on the question *"recent research in AI agent memory"* with the scope note *"focus on empirical evaluations since 2023."* Every role used `gpt-4o-mini`. The enabled sources were arXiv, OpenAlex and Crossref, reading was abstract-only, and the budget was 40 documents, 30 search operations, 2 snowball rounds and 3 iterations. Table 3 summarizes the run, and Table 4 traces it. About 3.5 minutes passed between the planner's output and the finished report.

> **What worked**
>
> The adaptive loop behaved as designed. After the first iteration, the critic found a dimension the planner had not anticipated (the effect of memory management on user trust and satisfaction) and added it as RQ6. RQ6 then received its own intents and evidence. In the second iteration, one paper (`doc_0139`) that had already been read for five subquestions was read again only for RQ6, and three more evidence items were extracted for it. This shows the per-pair accounting working: earlier document–subquestion pairs were not paid for again. Reading allocation spread the budget across subquestions, and every subquestion ended with at least seven evidence items. Canonicalization merged 34% of first-iteration hits (66 of 195), and 45 of 105 hits in the second iteration resolved to documents already in the corpus. The mean evidence strength of 0.59 sits right at the abstract-only cap, as expected when no experiment can be checked.

Table 3 Run summary.

Metric	Value
Research iterations	2
Search operations	30 / 30
Snowball rounds	2 / 2
LLM calls	116
Canonical documents	237
via search (depth 0)	189
via backward snowball	48
Documents read	28 / 40
Documents cited in report	22
Evidence items	87
experimental / survey	38 / 24
theoretical / benchmark	19 / 6
flagged as limitations	10
Mean evidence strength	0.59
Subquestions (final)	6
Mean coverage (critic)	0.75

Table 4 Condensed trace of the run.

It.	Event
1	5 subquestions planned, 9 intents, routed to 18 queries.
	195 raw hits → 129 canonical (66 merged).
	16 selected for reading (30 doc–SQ pairs).
	Snowball round 1: 4 backward seeds, 48 candidates, 48 new (ρ = 1.0).
	Snowball round 2: same 4 seeds, 48 candidates, 0 new (ρ = 0).
	14 read, 46 evidence items; coverage 0.75–0.85 per SQ.
	Critic: *continue*, coverage 0.80; `add` RQ6 (user trust and satisfaction), 4 intents.
2	4 new intents, 12 queries (3 intents closed), 105 hits → 60 new.
	15 selected; 14 read, 1 judged not relevant.
	87 evidence items; RQ3 coverage 0.90.
	Critic: *continue*; proposes `mark_answered` RQ3, `deprioritize` RQ4.
	Search budget used up ⇒ report.

What the run exposed

The case study also showed six weaknesses, which we report because they are informative.

1. **Repeated snowball seeds.** The second round chose the *same* four seeds in the same direction and spent a round fetching 48 edges that were already in the corpus. The seed selector excludes snowballed documents as seeds but not documents that have already been expanded. The saturation rule needs two low-yield rounds, so the round cap ended snowballing, not saturation. Excluding (seed, direction) pairs that have already been expanded would fix this.
2. **Inflated intent-yield counts.** When RQ6 was added, all 237 documents were re-screened against it, and each re-screened document credited its originating intents again. One intent therefore reports 18 relevant documents out of 15 found. The critic reads these yields when judging saturation, so the counts should be de-duplicated per (intent, document) pair.
3. **No arXiv hits.** arXiv returned zero hits for every query in both iterations, while OpenAlex and Crossref returned 135 and 60 in the first. The literal, AND-combined matching suggests the LLM-generated arXiv queries were too specific, but we have not confirmed the cause. Either way, a source that returns nothing still costs search budget.
4. **Weak corroboration signal.** Only 2 of 237 canonical documents were returned by more than one source, so the multi-source-corroboration term in Equation (1) was nearly constant. With only two sources returning results, this signal adds little.
5. **Unused final critique.** The second critique's edits (close RQ3, deprioritize RQ4) were recorded but not applied, because the search budget stopped the run first. This follows from the termination order, but the report should present these edits as recommended next steps.
6. **Misleading message on failure.** In an earlier run where the model provider's API key was not loaded, every LLM call failed and each stage fell back to its default. The run completed and produced a diagnostic report instead of crashing, but it gave its stopping reason as "the research critic judged the evidence sufficient." The fallback query also passed the user's raw question, including a trailing "?", to OpenAlex, which rejected it as an invalid wildcard. Both problems are small, but they make diagnosis harder exactly when it is needed.

14 Discussion and Limitations

Evaluation. This paper describes a system and does not evaluate it in a controlled way. We have not yet measured recall against human-curated reviews, the precision of extracted claims, or the contribution of each component, such as decomposition versus a single query or snowballing versus none. The state records what such ablations need, including per-intent yield, the provenance of every document, and the snowball depth at which each was found. Running them is the most important next step.

Abstract-only evidence. The default reading mode extracts evidence from abstracts. It is fast and cheap, but it limits how strong any claim can be, and the evidence store reflects this honestly. Full-text reading is supported but has not yet been evaluated at scale.

Claim reconciliation. Exact-text claim clustering underestimates how many documents support a claim, because paraphrased claims form separate clusters. A concept graph or embedding-based clustering layer would fit naturally into the evidence store and would make the weak-support signal more reliable.

LLM judgement remains the bottleneck. Deterministic machinery cannot recover from screening or extraction errors. In our model tests, the most damaging errors were those that general benchmarks do not measure: rejecting a clearly relevant paper, and copying examples from the prompt as search intents. Model choice for the screener and reader roles should be validated on the system's own schemas.

15 Conclusion

litresearch treats literature research as iterative evidence acquisition over an explicit, mutable state. In this setting the research plan is an output of the investigation as well as an input, the result of reading is structured evidence rather than prose, and everything that does not require judgement is deterministic code. This design makes the system auditable: every report sentence can be traced to a claim, every claim to a document, and every document to the queries and intents that found it. The same traceability made the weaknesses in our case study visible, including repeated snowball seeds, inflated intent yields and silent sources. We expect this property to be as useful when improving the system as when using it.

References

[1] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'Arcy, et al. OpenScholar: Synthesizing scientific literature with retrieval-augmented LMs. *arXiv preprint arXiv:2411.14199*, 2024.

[2] Ahmed K. Elmagarmid, Panagiotis G. Ipeirotis, and Vassilios S. Verykios. Duplicate record detection: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 19(1):1–16, 2007.

[3] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.

[4] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[5] Rodney Kinney, Chloe Anastasiades, Russell Authur, Iz Beltagy, Jonathan Bragg, Alexandra Buraczynski, et al. The Semantic Scholar open data platform. *arXiv preprint arXiv:2301.10140*, 2023.

[6] Barbara Kitchenham. Procedures for performing systematic reviews. Technical Report TR/SE-0401, Keele University, 2004.

[7] Jakub Lála, Odhran O'Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G. Rodriques, and Andrew D. White. PaperQA: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559*, 2023.

[8] LangChain, Inc. LangGraph: Building stateful, multi-actor applications with LLMs. `https://github.com/langchain-ai/langgraph`, 2024. Software library.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[10] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[11] Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, 372:n71, 2021.

[12] Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv preprint arXiv:2205.01833*, 2022.

[13] Yijia Shao, Yucheng Jiang, Theodore A. Kanell, Peter Xu, Omar Khattab, and Monica S. Lam. Assisting in writing Wikipedia-like articles from scratch with large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.

[14] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[15] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

[16] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[17] Claes Wohlin. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, 2014.

[18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.